Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
Wydział Informatyki

Algorytmy macierzowe

Rekurencyjna faktoryzacja LU

Metody Bineta, Strassena i Laderman — sprawozdanie z zadania 2

Piotr Szamocki · Mateusz Stoch · Patrycja Piasecka

Kraków, listopad 2026

Spis treści

1. Cel zadania 2
2. Język i środowisko 2
3. Implementacja 3
4. Pseudokody 4
5. Najważniejsze fragmenty kodu 5
6. Wyniki pomiarów 6
7. Poprawność algorytmu 9
8. Wnioski 11

1. Cel zadania

Zaimplementowaliśmy rekurencyjną faktoryzację LU macierzy kwadratowej, $A = LU$. L jest dolnotrójkątna z jedynkową przekątną, U jest górnotrójkątna. Wszystkie mnożenia macierzy w algorytmie wykonuje jedna z metod z zadania 1:

1. metoda Bineta (bloki 2×2, 8 mnożeń bloków),
2. metoda Strassena (bloki 2×2, 7 mnożeń bloków),
3. metoda Laderman (bloki 3×3, 23 mnożenia bloków) — dodatek ponad wymagania polecenia.

Wymagania z polecenia:

- macierze kwadratowe o dowolnym rozmiarze,
- elementy losowe z przedziału $(10^{-8}, 1)$,
- zliczanie operacji zmiennoprzecinkowych $(+ - * /)$,
- mnożenia macierzy wykonują nasze algorytmy z zadania 1.

Każdy wariant ma wersję jednowątkową i wielowątkową. Obie wykonują te same operacje.

1.1. Macierze testowe i brak pivotowania

Faktoryzacja jest bez pivotowania. Na zwykłej macierzy losowej z $(10^{-8}, 1)$ takie LU jest niestabilne: dla $n = 2500$ największy element U ma około $1.2 \cdot 10^5$, a wynik Strassena różni się od klasycznego LU nawet o 2% (względnie). Dlatego w pomiarach elementy są losowe z $(10^{-8}, 1)$, a do każdego elementu przekątnej dodajemy n. Macierz jest wtedy ściśle diagonalnie dominująca, więc LU bez pivotowania istnieje i jest stabilne. Elementy poza przekątną pochodzą z przedziału z polecenia. Dla macierzy z zerowym minorem głównym program zgłasza `ArithmeticException`.

2. Język i środowisko

2.1. Dlaczego Java nie jest językiem interpretowanym

Kompilator `javac` tłumaczy kod źródłowy na kod bajtowy. Podczas działania maszyna wirtualna (HotSpot) kompiluje często używane fragmenty do kodu maszynowego (kompilacja JIT). Pętle faktoryzacji i mnożenia wykonują się więc jako kod maszynowy.

Konsekwencja: pierwsze wywołania są wolniejsze, bo kod nie jest jeszcze skompilowany. Dlatego powtarzamy pomiary i uśredniamy wyniki (Rozdział 6.1).

2.2. Sprzęt

Tabela 1: Konfiguracja pomiarowa

Procesor	Intel Core i7-13700HX (16 rdzeni, 24 wątki)
Pamięć RAM	32 GB
System operacyjny	Windows 10 Home
JDK	OpenJDK 25
Maks. sterta JVM	8,5 GB (domyślnie, 1/4 RAM)

3. Implementacja

Z zadania 1 używamy bez zmian widoków na tablicę 1D (podział na bloki bez kopiowania danych), algorytmów mnożenia Bineta, Strassena i Laderman (ze Strassenem i Laderman dla dowolnych rozmiarów dzięki peelingowi) oraz puli `ForkJoinPool`. Nowe są: rekurencja LU, rekurencyjne rozwiązywanie układów trójkątnych i kernele klasyczne na liściach rekursji.

3.1. Algorytm blokowy

Dzielimy macierz na cztery bloki, A_{11} ma $\lfloor n/2 \rfloor$ wierszy:

$$\left(\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right) = \left(\begin{array}{c|c} L_{11} & 0 \\ \hline L_{21} & L_{22} \end{array}\right) \left(\begin{array}{c|c} U_{11} & U_{12} \\ \hline 0 & U_{22} \end{array}\right). \tag{1}$$

Mnożąc bloki, dostajemy cztery równania:

$$A_{11} = L_{11}U_{11}, \quad A_{12} = L_{11}U_{12}, \quad A_{21} = L_{21}U_{11}, \quad A_{22} = L_{21}U_{12} + L_{22}U_{22}. \tag{2}$$

Stąd kolejność obliczeń:

1. $L_{11}U_{11} = A_{11}$ — rekurencyjna faktoryzacja LU,
2. $U_{12} = L_{11}^{-1}A_{12}$ — układ trójkątny z wieloma prawymi stronami,
3. $L_{21} = A_{21}U_{11}^{-1}$ — układ trójkątny z lewej strony,
4. $S = A_{22} - L_{21}U_{12}$ — jedno mnożenie macierzy i odejmowanie (dopełnienie Schura),
5. $L_{22}U_{22} = S$ — rekurencyjna faktoryzacja LU.

Układy z punktów 2 i 3 też rozwiązujemy rekurencyjnie (Algorytm 2, Algorytm 3). Cała praca poza liśćmi to mnożenia macierzy, więc zamiana algorytmu mnożenia zmienia koszt faktoryzacji.

Faktoryzacja działa w miejscu. Program kopiuje A do macierzy wynikowej i nadpisuje ją: L leży poniżej przekątnej (jedynki z przekątnej nie są przechowywane), U leży na przekątnej i nad nią.

3.2. Zliczanie operacji

Program zlicza operacje podczas działania. Dzielenia liczymy jako mnożenia, odejmowania jako dodawania. Kernele liściowe zliczają operacje wzorami, raz na wywołanie (Tabela 2). Mnożenia i odejmowania bloków liczą się jak w zadaniu 1.

Tabela 2: Koszt kerneli klasycznych (L, U: $n \times n$; prawa strona ma m kolumn albo wierszy)

Działanie	**Mnożenia**	**Dodawania**
LU klasyczne $n \times n$	$(n^3 - n)/3$	$(n-1)n(2n-1)/6$
$B \coloneqq L^{-1}B$, B: $n \times m$	$mn(n-1)/2$	$mn(n-1)/2$
$B \coloneqq BU^{-1}$, B: $m \times n$	$mn(n-1)/2 + mn$	$mn(n-1)/2$

W BU^{-1} dodatkowe mn mnożeń to dzielenia przez elementy przekątnej U.

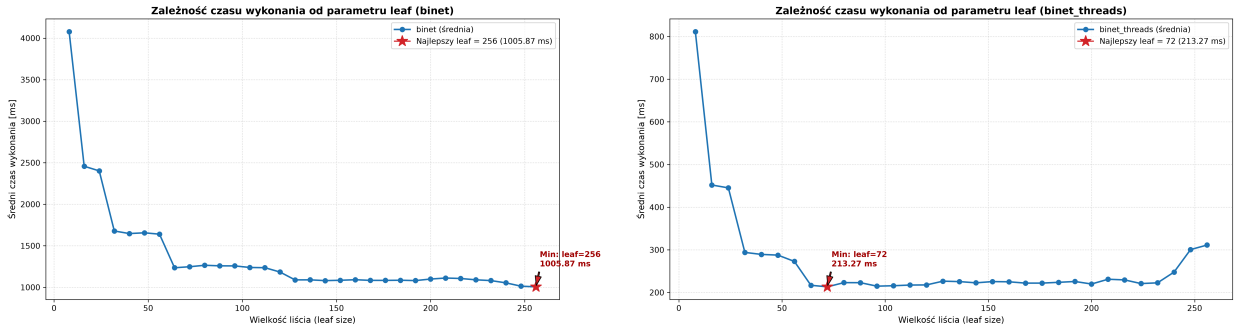
3.3. Wersja jedno- i wielowątkowa

- **Jednowątkowa** — wszystko wykonuje się po kolei.
- **Wielowątkowa** — `ForkJoinPool` z liczbą wątków równą liczbie wątków procesora (24). Równolegle idą wnętrza mnożeń (jak w zadaniu 1) oraz para układów trójkątnych U_{12} i L_{21}, które nie zależą od siebie. Reszta jest sekwencyjna, bo każdy krok potrzebuje wyniku poprzedniego.

3.4. Próg rekursji

Rekursja kończy się, gdy $n \le n_0$ (parametr `leaf`) albo $n < 2$. Wtedy używamy kernela klasycznego: LU klasycznego, a w układach trójkątnych podstawień w przód i w tył. Ten sam n_0 dostają algorytmy mnożenia.

Wpływ progu zmierzyliśmy dla Bineta: $n_0 = 8, 16, ..., 256$, rozmiary $n = 1900, 1910, ..., 1990$, 3 przebiegi (Rysunek 1). Użyliśmy macierzy diagonalnie dominujących. Czas nie zależy od wartości elementów, bo liczba i rodzaj operacji są takie same.



Rysunek 1: Średni czas LU metodą Bineta w zależności od n_0: wersja jednowątkowa (po lewej) i wielowątkowa (po prawej)

W wersji wielowątkowej czas spada do $n_0 \approx 64$, potem jest prawie stały (215–230 ms) i rośnie dla $n_0 \geq 240$. W wersji jednowątkowej czas maleje do końca badanego zakresu (1006 ms dla $n_0 = 256$). Jedna wartość n_0 musi obsłużyć obie wersje, żeby wykonywały te same operacje. Wybraliśmy $n_0 = 176$: ma najmniejszą sumę czasów obu wersji, każdego podzielonego przez jego minimum. Daje 1083 ms w wersji jednowątkowej (najlepszy: 1006 ms) i 222 ms w wielowątkowej (najlepszy: 213 ms). Wszystkie pomiary w sprawozdaniu wykonano z $n_0 = 176$.

4. Pseudokody

Notacja. Bloki to widoki na macierz, zapis do bloku trafia od razu do macierzy. Iloczyny $L_{21}B_1$ i $A_{21}A_{12}$ wykonuje dowolny algorytm mnożenia z zadania 1 (Binet albo Strassen). W bloku **równolegle** oba zadania wykonują się na puli wątków w wersji wielowątkowej, a po kolei w wersji jednowątkowej.

4.1. Faktoryzacja LU

$[A] := \mathrm{LU}(A)$ ▷ w miejscu: $A := L \setminus U$

jeśli $n \leq n_0$ **lub** $n < 2$ **to**
  $A := \mathrm{LU}_{\text{klasyczne}}(A)$
  zwróć
koniec jeśli

$$A \to \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right)$$

gdzie A_{TL} ma $\lfloor n/2 \rfloor$ wierszy i kolumn

$A_{TL} := \mathrm{LU}(A_{TL})$ ▷ $L_{11} \setminus U_{11}$
równolegle
  $A_{TR} := \mathrm{SolveL}(A_{TL}, A_{TR})$ ▷ $U_{12} = L_{11}^{-1} A_{12}$ (Algorytm 2)
  $A_{BL} := \mathrm{SolveU}(A_{TL}, A_{BL})$ ▷ $L_{21} = A_{21} U_{11}^{-1}$ (Algorytm 3)
koniec równolegle
$T := A_{BL} A_{TR}$ ▷ mnożenie macierzy
$A_{BR} := A_{BR} - T$ ▷ dopełnienie Schura
$A_{BR} := \mathrm{LU}(A_{BR})$ ▷ $L_{22} \setminus U_{22}$

Algorytm 1: Rekurencyjna faktoryzacja LU

4.2. Układy trójkątne

Oba algorytmy rozwiązują układ w miejscu. Rekurencja idzie po macierzy trójkątnej, a prawa strona jest dzielona razem z nią.

$[B] := \mathrm{SolveL}(L, B)$ ▷ $B := L^{-1}B$, L dolnotrójkątna z jedynkową przekątną

jeśli $n \le n_0$ **lub** $n < 2$ **to**
  $B := L^{-1}B$ ▷ podstawianie w przód
  zwróć
koniec jeśli

$$L \to \left(\begin{array}{c|c} L_{TL} & 0 \\ \hline L_{BL} & L_{BR} \end{array}\right), B \to \begin{pmatrix} B_T \\ \hline B_B \end{pmatrix}$$

$B_T := \mathrm{SolveL}(L_{TL}, B_T)$
$T := L_{BL}B_T$ ▷ mnożenie macierzy
$B_B := B_B - T$
$B_B := \mathrm{SolveL}(L_{BR}, B_B)$

Algorytm 2: Układ z macierzą dolnotrójkątną

$[B] := \mathrm{SolveU}(U, B)$ ▷ $B := BU^{-1}$, U górnotrójkątna

jeśli $n \le n_0$ **lub** $n < 2$ **to**
  $B := BU^{-1}$ ▷ podstawianie, dzielenie przez przekątną
  zwróć
koniec jeśli

$$U \to \left(\begin{array}{c|c} U_{TL} & U_{TR} \\ \hline 0 & U_{BR} \end{array}\right), B \to (B_L \,|\, B_R)$$

$B_L := \mathrm{SolveU}(U_{TL}, B_L)$
$T := B_L U_{TR}$ ▷ mnożenie macierzy
$B_R := B_R - T$
$B_R := \mathrm{SolveU}(U_{BR}, B_R)$

Algorytm 3: Układ z macierzą górnotrójkątną z prawej strony

5. Najważniejsze fragmenty kodu

5.1. Krok rekurencyjny LU

Blok A_{11} jest faktoryzowany, potem dwa niezależne układy trójkątne (jeden w puli, drugi w bieżącym wątku), mnożenie i odejmowanie, rekurencja na dopełnieniu Schura.

```
int n1 = n / 2, n2 = n - n1;
var a11 = a.block(0, 0, n1, n1);   var a12 = a.block(0, n1, n1, n2);
var a21 = a.block(n1, 0, n2, n1);  var a22 = a.block(n1, n1, n2, n2);

factorRec(ops, mul, a11, leaf, pool);
Runnable u12 = () -> solveLower(ops, mul, a11, a12, leaf, pool);       // U12 = L11^-1 A12
Runnable l21 = () -> solveUpperRight(ops, mul, a11, a21, leaf, pool);  // L21 = A21 U11^-1
if (pool != null) {
    var f = ForkJoinTask.adapt(u12).fork();
    l21.run();
    f.join();
} else {
    u12.run();
    l21.run();
}
var t = ops.newMatrix(n2, n2).view();
```

```
mul.multiply(ops, a21, a12, t, leaf, pool);   // T = L21 U12 (Binet / Strassen / Laderman)
ops.subtract(a22, t, a22);                    // S = A22 - T
factorRec(ops, mul, a22, leaf, pool);
```

5.2. Układ z macierzą dolnotrójkątną

Dolna połowa prawej strony jest poprawiana po rozwiązaniu górnej. Układ BU^{-1} jest zbudowany symetrycznie (podział B na kolumny).

```
var b1 = b.block(0, 0, n1, m);
var b2 = b.block(n1, 0, n2, m);

solveLower(ops, mul, l.block(0, 0, n1, n1), b1, leaf, pool);
var t = ops.newMatrix(n2, m).view();
mul.multiply(ops, l.block(n1, 0, n2, n1), b1, t, leaf, pool);
ops.subtract(b2, t, b2);
solveLower(ops, mul, l.block(n1, n1, n2, n2), b2, leaf, pool);
```

5.3. LU klasyczne (liście rekursji)

Kernel nadpisuje macierz w miejscu: mnożniki trafiają pod przekątną, a reszta jest aktualizowana.

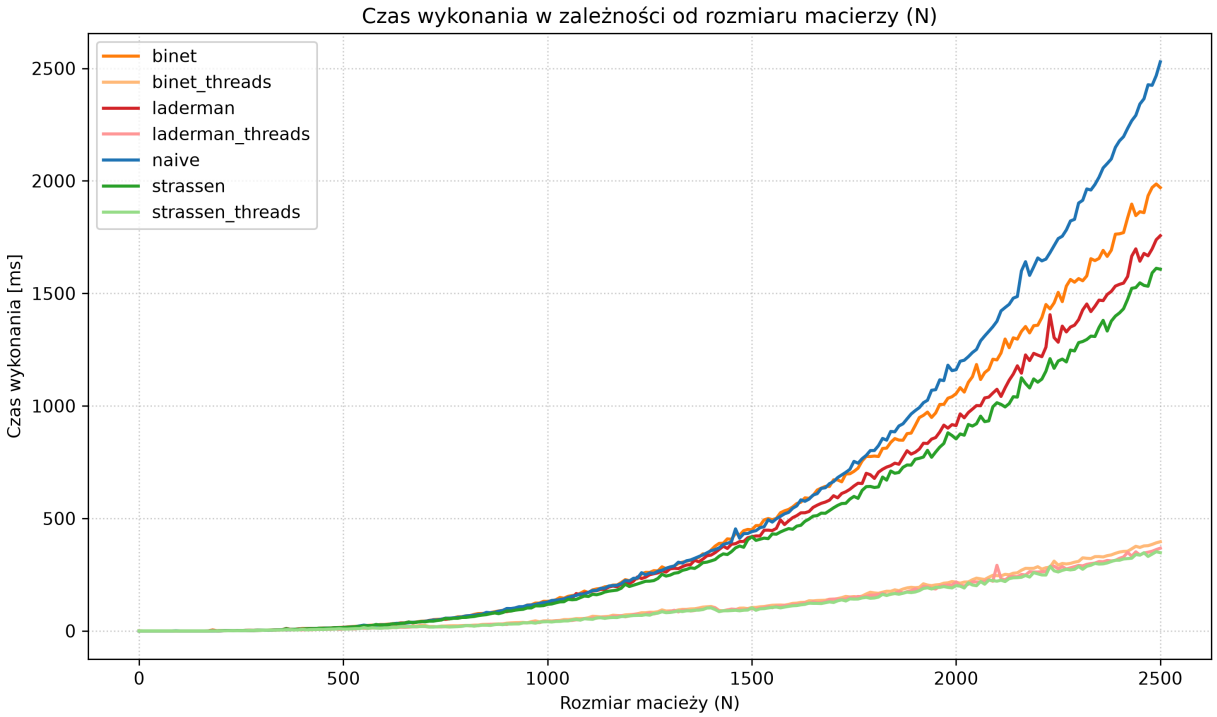
```
protected void luInPlaceImpl(MatrixView<double[]> a) {
    int n = a.rows;
    for (int k = 0; k < n; k++) {
        int top = a.idx(k, 0);
        double p = a.data[top + k];
        if (p == 0.0)
            throw new ArithmeticException("Zerowy element główny (brak pivotowania)");
        for (int i = k + 1; i < n; i++) {
            int row = a.idx(i, 0);
            double f = a.data[row + k] / p;
            a.data[row + k] = f;
            for (int j = k + 1; j < n; j++)
                a.data[row + j] -= f * a.data[top + j];
        }
    }
}
```

6. Wyniki pomiarów

6.1. Metodyka

- Rozmiary: $n = 10, 20, ..., 2500$ (krok 10).
- Macierze: elementy losowe z $(10^{-8}, 1)$, do przekątnej dodane n (Rozdział 1.1). To samo ziarno dla wszystkich wariantów.
- Próg rekursji: $n_0 = 176$ (Rozdział 3.4).
- Każdy wariant: czas, liczba mnożeń, liczba dodawań.
- 3 przebiegi, czasy uśrednione. Liczby operacji są deterministyczne.
- Wersje wielowątkowe: 24 wątki (oznaczone `_threads`). Klasyczne LU (`naive`) jest punktem odniesienia.

6.2. Czas działania

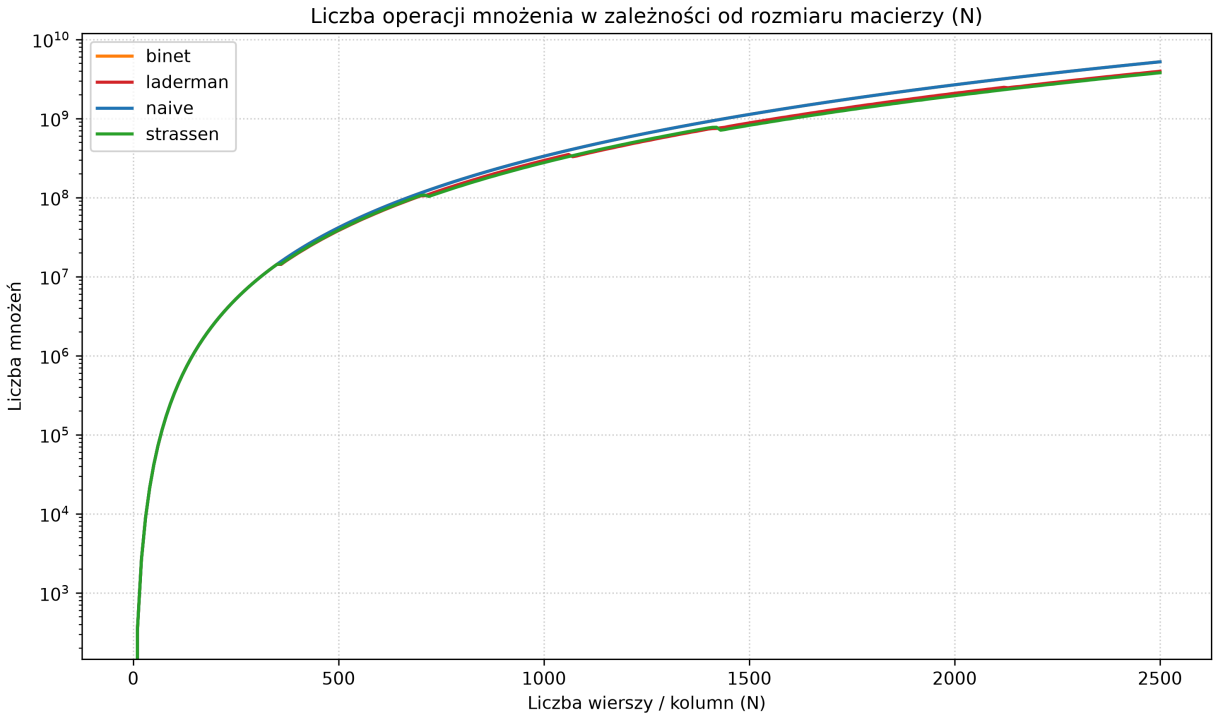


Rysunek 2: Czas faktoryzacji LU macierzy $n \times n$

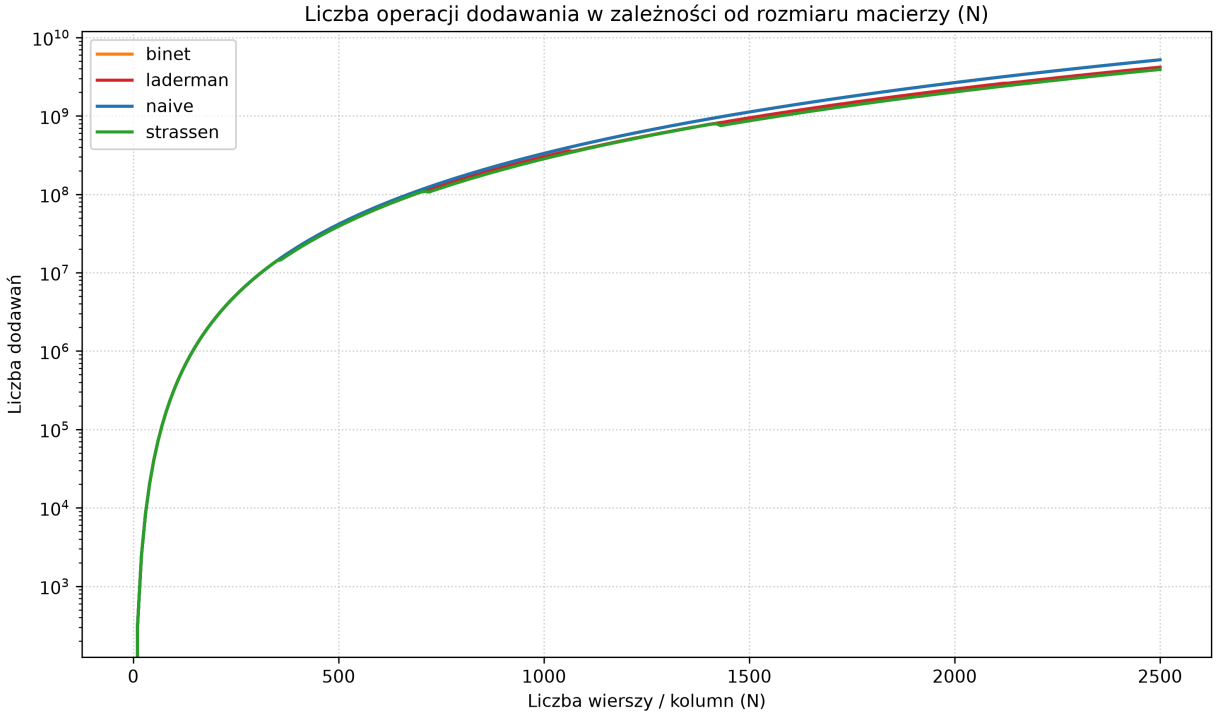
Do $n \approx 1000$ różnice między wariantami jednowątkowymi są małe, od $n \approx 1500$ rosną. Dla $n = 2500$ klasyczne LU potrzebuje 2529 ms, Binet 1970 ms (o 22% mniej), Strassen 1607 ms (o 36% mniej, czyli 1,57 razy szybciej), a Laderman 1756 ms (o 31% mniej, 1,44 razy szybciej). Binet wykonuje tyle samo operacji co wersja klasyczna, więc przewaga bierze się prawdopodobnie z lepszej lokalności pamięci: rekursja pracuje na małych blokach.

Wersje wielowątkowe są kilka razy szybsze od jednowątkowych tej samej metody: dla $n = 2500$ Binet potrzebuje 396 ms (4,97 razy szybciej), Strassen 348 ms (4,62 razy szybciej), a Laderman 368 ms (4,78 razy szybciej). Przyspieszenie jest dużo mniejsze od liczby wątków (24). Szkielet LU jest sekwencyjny, a kernele liściowe działają w jednym wątku.

6.3. Liczba operacji



Rysunek 3: Liczba mnożeń zmiennoprzecinkowych (skala logarytmiczna)



Rysunek 4: Liczba dodawań zmiennoprzecinkowych (skala logarytmiczna)

Wersje jedno- i wielowątkowa wykonują te same operacje, więc rysujemy jedną krzywą na metodę. Klasyczne LU wykonuje $(n^3 - n)/3$ mnożeń i $(n-1)n(2n-1)/6$ dodawań, czyli około $n^3/3$. Dla $n = 2500$ wzory dają 5 208 332 500 mnożeń i 5 205 208 750 dodawań, dokładnie tyle zliczył program. Binet wykonuje identyczną liczbę operacji we wszystkich zmierzonych rozmiarach (krzywe się pokrywają), bo rekursja zmienia tylko kolejność iloczynów $l_{ik}u_{kj}$, a nie ich zbiór.

Strassen wykonuje mniej mnożeń od $n > n_0$. Dla $n = 2500$ ma 3 782 919 315 mnożeń i 3 914 660 800 dodawań, czyli 72,6% i 75,2% wartości klasycznych. Laderman ma 3 960 207 500 mnożeń i 4 197 676 438 dodawań (76,0% i 80,6%). Oszczędność dodawań jest mniejsza, bo obie metody dodają bloki; Laderman potrzebuje ich więcej niż Strassen.

6.4. Maksymalny rozmiar macierzy

Sprawdziliśmy, jak dużą macierz da się zfaktoryzować przy domyślnej stercie JVM (8,5 GB, Tabela 1). Szukaliśmy największego n, dla którego program kończy się bez `OutOfMemoryError`: podwajanie n od 2500 do pierwszego niepowodzenia, potem wyszukiwanie binarne z dokładnością do 100. Każda próba to osobny proces JVM, wersja wielowątkowa, $n_0 = 176$ (Tabela 3).

Tabela 3: Największy rozmiar macierzy mieszczący się w pamięci (sterta 8,5 GB)

Metoda	**Maks. n**	**Pierwszy n z brakiem pamięci**
Binet	21 328	21 406
Strassen	11 796	11 875
Laderman	11 562	11 640

Binet trzyma w pamięci tylko macierz wejściową i wynik, po $8n^2$ bajtów każda: dla $n = 21328$ to 7,3 GB z 8,5 GB sterty. Strassen potrzebuje jeszcze buforów na iloczyny pośrednie $M_1, ..., M_7$ i sumy argumentów, które istnieją jednocześnie z macierzami, więc jego granica jest niższa. Laderman ma podobne bufory (iloczyny pośrednie 23 bloków) i mieści się w nieco mniejszych macierzach niż Strassen. Liczyliśmy tylko wersje wielowątkowe, bo bufory są przydzielane z góry i zużycie pamięci powinno być takie samo w wersjach jednowątkowych.

7. Poprawność algorytmu

Dla każdego rozmiaru w pomiarach program porównuje wynik każdego wariantu z klasycznym LU liczonym na całej macierzy (element po elemencie, tolerancja względna 10^{-9}). Wszystkie warianty przeszły to porównanie. Poniżej pokazujemy dodatkowo przykład dla małej macierzy i porównanie z Octave.

7.1. Przykład dla macierzy 4×4

Wylosowaliśmy A o rozmiarze 4×4 z przedziału $(10^{-8}, 1)$ (bez dodawania n do przekątnej). Wykonaliśmy faktoryzację każdą metodą z $n_0 = 1$, w wersji jedno- i wielowątkowej, żeby rekursja wykonała się do końca. Wynik programu (zaokrąglony do 4 miejsc):

$$A = \begin{pmatrix} 0.6186 & 0.8492 & 0.7904 & 0.5296 \\ 0.8417 & 0.1483 & 0.8438 & 0.5848 \\ 0.1667 & 0.7158 & 0.5393 & 0.2795 \\ 0.0557 & 0.0131 & 0.3578 & 0.7500 \end{pmatrix}, \tag{3}$$

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1.3605 & 1 & 0 & 0 \\ 0.2695 & -0.4835 & 1 & 0 \\ 0.0900 & 0.0629 & 1.4057 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 0.6186 & 0.8492 & 0.7904 & 0.5296 \\ 0 & -1.0071 & -0.2316 & -0.1358 \\ 0 & 0 & 0.2143 & 0.0711 \\ 0 & 0 & 0 & 0.6109 \end{pmatrix}. \tag{4}$$

Pierwszy krok sprawdzony ręcznie: $l_{21} = a_{21}/a_{11} = 0.8417/0.6186 = 1.3605$, $u_{22} = a_{22} - l_{21}a_{12} = 0.1483 - 1.3605 \cdot 0.8492 \approx -1.007$. Zgadza się z macierzami wyżej.

7.2. Porównanie z referencyjnym LU

Procedura:

1. Zapisaliśmy A i spakowane $L \setminus U$ z każdego wariantu z pełną precyzją (`%.17g`).
2. Policzyliśmy referencyjne LU bez pivotowania (ta sama kolejność działań co w programie) i porównaliśmy z wynikami programu.

```
A = load('A.txt');  n = rows(A);
L = eye(n); U = A;
for k = 1:n-1
  for i = k+1:n
    L(i,k) = U(i,k) / U(k,k);
    U(i,k:n) = U(i,k:n) - L(i,k) * U(k,k:n);
  end
end
U = triu(U);
M = load('LU_binet.txt');  Lj = tril(M,-1) + eye(n);  Uj = triu(M);
max(max(abs(L - Lj))),  max(max(abs(U - Uj))),  max(max(abs(Lj*Uj - A)))
```

Tabela 4: Porównanie z referencyjnym LU bez pivotowania, macierz 4×4, $n_0 = 1$ ($\varepsilon \approx 2.2 \cdot 10^{-16}$)

Metoda	**Mnożenia / dodawania**	$\max\lvert L - L_j \rvert$	$\max\lvert U - U_j \rvert$	$\max\lvert L_j U_j - A \rvert$
Binet	20 / 14	0	0	$1.1 \cdot 10^{-16}$
Strassen	19 / 28	0	$1.1 \cdot 10^{-16}$	$1.1 \cdot 10^{-16}$
Laderman	20 / 14	0	0	$1.1 \cdot 10^{-16}$

Wersje jedno- i wielowątkowe dały identyczne wyniki. Binet i Laderman zgadzają się z referencją co do bitu (dla $n = 4$ Laderman dzieli macierz tak, że wykonuje te same operacje co Binet, więc ten przykład nie sprawdza jego bloków 3×3; robią to testy na dużych macierzach), Strassen różni się o jedną jednostkę ostatniego miejsca w U, bo wykonuje więcej dodawań i odejmowań. Reszta $L_j U_j - A$ jest rzędu ε. Dla $n = 4$ Binet wykonuje tyle samo operacji co klasyczne LU ($(4^3 - 4)/3 = 20$ mnożeń i $3 \cdot 4 \cdot 7/6 = 14$ dodawań). Wyniki są poprawne.

8. Wnioski

1. Faktoryzację LU można zapisać blokowo tak, że cała praca poza liśćmi to mnożenia macierzy, bo układy trójkątne też rozwiązujemy przez mnożenia. Zamiana algorytmu mnożenia zmienia więc koszt faktoryzacji.
2. Binet wykonuje dokładnie tyle samo operacji co klasyczne LU (około $n^3/3$ mnożeń i dodawań), a dla $n = 2500$ jest od niego o 22% szybszy.
3. Strassen wykonuje dla $n = 2500$ 72,6% mnożeń i 75,2% dodawań wersji klasycznej. Jest najszybszą metodą: 1607 ms wobec 2529 ms dla klasycznego LU. Laderman (76,0% mnożeń, 80,6% dodawań) daje 1756 ms, czyli jest między Strassenem a Binetem.
4. Wersje wielowątkowe (`ForkJoinPool`) dają przyspieszenie 4,6–5,0 razy dla $n = 2500$.
5. Strassen potrzebuje dodatkowych buforów, więc przy domyślnej stercie 8,5 GB mieści macierze do $n \approx 11800$, a Binet do $n \approx 21300$ (Laderman do $n \approx 11560$).
6. Wyniki zgadzają się z referencją z dokładnością do błędu zaokrągleń (około ε dla macierzy 4×4).
7. Faktoryzacja nie używa pivotowania, więc wymaga macierzy z niezerowymi minorami głównymi. Na zwykłych macierzach losowych jest niestabilna (Rozdział 1.1), dlatego testowaliśmy macierze diagonalnie dominujące.